

qmark

Cisco-style ? help, right in your terminal

AI explanations in plain English — before you run the command.

The problem

- `man` pages are complete but **overwhelming**
- `--help` output is dense and **inconsistent** between tools
- New users copy-paste commands they **don't understand**

You shouldn't need to leave the terminal to learn the terminal.

The idea

Cisco solved discoverability decades ago with **one keystroke**:

```
Router(config)# ip ?  
  address      Set the IP address of an interface  
  route        Establish static routes  
  ...
```

qmark brings that experience to your everyday shell.

What it looks like

Type a command, press `?`, and an interactive picker opens on the spot:

```
$ git ?
— qmark — help for `git` — ↑↓ move · ↵ insert · Esc close · 3/24
clone      Clone a repository into a new directory
init       Create an empty Git repository or reinitialize an existing one
add        Add file contents to the index
mv         Move or rename a file, a directory, or a symlink
restore    Restore working tree files
...
$ git add █
```

↑↓ to move, Enter to insert the choice, Esc to leave the line untouched.

Subcommand-aware

? answers for the command you are **actually** typing, not just its base:

```
$ git mv ?  
— qmark — help for `git mv` —————  
-v, --[no-]verbose    be verbose  
-n, --[no-]dry-run    dry run  
-f, --[no-]force      force move/rename even if target exists  
-k                    skip move/rename errors  
--[no-]sparse        allow updating entries outside of the sparse-checkout cone
```

It tries `git mv --help`, then `git mv -h`, then falls back to `git --help`.

AI explanations — v0.2

The CLI surface is frozen early, so today `explain` tells the truth:

```
$ qmark explain "tar -xzvf archive.tar.gz -C /tmp"  
— qmark — explain _____
```

```
tar -xzvf archive.tar.gz -C /tmp
```

```
The AI backend is not wired up yet (this is the scaffold release).  
Once it lands, set QMARK_AI_PROVIDER and QMARK_AI_API_KEY to enable it.
```

The goal for v0.2: one plain-English sentence, before you run it — provider-agnostic (Anthropic, OpenAI-compatible, local Ollama), with an offline cache. Only the command line is ever sent. Never your history, environment or files.

Features

- **? at the prompt** — contextual help wired into zsh and bash
- **Interactive picker** — arrow keys, Enter inserts, Esc cancels
- **Subcommand-aware** — `git mv ?` shows `git mv`'s flags
- **Curated fallbacks** — tools whose help is unparseable still answer
- **qmark suggest** — the same engine, callable directly
- **Single static binary** — written in Rust, no runtime required

How it works

```
you type `git mv ` and press ?  
  |      ZLE widget (zsh) · bind -x (bash)  
  ▼  
qmark suggest --interactive -- "git mv "  
  |      harvests `git mv --help`, parses the option columns  
  ▼  
interactive picker, drawn on /dev/tty  
  |      Enter prints the chosen entry on stdout – and nothing else  
  ▼  
the shell splices it into the line:  git mv --dry-run
```

Thin shell snippets, one small binary. Details in `docs/ARCHITECTURE.md`.

Rules of the `?` key

A shell can't intercept `?` unconditionally — it is a glob character:

- `?` opens help **only** at the end of a line that ends with a space
- `ls file?.txt` inserts a literal `?` — **globs keep working**
- `qmark` not on `PATH` → plain `?`. Your prompt never breaks because of us
- `QMARK_NO_BIND=1` loads the functions without binding the key

These rules are a public contract: changing them needs an issue first.

Getting started

```
git clone https://github.com/Vitruvi0/qmark.git
cd qmark
cargo install --path .

# ~/.zshrc or ~/.bashrc
eval "$(qmark init zsh)" # or: qmark init bash
```

Then type a command, a space, and **?**.

Roadmap

- **v0** — repo scaffold, CLI skeleton, zsh/bash `?` binding, interactive subcommand-aware picker
- **v0.2** — AI backend for `explain`, offline cache, config file
- **v0.3** — PowerShell support, tldr pages, man parsing

Status: **pre-alpha**, under active development.

APIs, commands and key bindings will change without notice.

Try it. Type `?`.

github.com/Vitruvi0/qmark — MIT licensed

Contributions welcome once public — see `CONTRIBUTING.md`.